Community Resilience and Everyday Services Platform

Project idea

v.2026.09.08

Disclaimer
(1) Parts of this document have been generated using AI models. *(2) This is to be consider a "seed" document for each team to develop its own product*

1 Project Overview

Communities depend on digital services to coordinate activities, share reliable information, manage resources, and respond to disruptions. These communities may include municipalities, university campuses, neighbourhoods, residential complexes, company campuses, voluntary organizations, or other groups with shared needs.

In this laboratory project, each team will design and develop a digital product that helps a community **anticipate, manage, or recover from disruptions affecting everyday life**.

Examples of disruptions include:

- Transport interruptions
- Extreme weather events
- Facility or service closures
- Accessibility barriers
- Energy or resource constraints
- Equipment failures
- Changes to scheduled activities
- Temporary shortages
- Community events and crowd-management situations
- Requests for local assistance

Each team will operate as a **startup**, while the teaching staff will act as investors and product stakeholders. Teams are expected to identify a meaningful problem, investigate existing solutions, validate their assumptions, propose an innovative product, and deliver a credible Minimum Viable Product.

The project is not based on a fixed list of functional requirements. Teams must make and justify product, architectural, and technological decisions based on evidence gathered during the project.

The central project challenge is:

Create an investable digital product that strengthens the resilience of a real community by combining trustworthy information, coordinated action, responsible AI, and a secure, observable, continuously delivered software architecture.

The project will be executed in three sprints:

1. Sprint 1: Idea gathering, competitor analysis, product vision, and technical design.
2. Sprint 2: Development of the Minimum Viable Product (MVP).
3. Sprint 3: Completion and refinement of the MVP, with final presentations.

2 Learning objectives

By completing this project, students should demonstrate their ability to:

1. Discover and validate a meaningful product opportunity.
2. Translate stakeholder needs into a coherent product vision and prioritized backlog.
3. Design an appropriate software architecture and justify its principal trade-offs.
4. Develop and integrate frontend, backend, data, and AI components.
5. Apply agile product and project management practices.
6. Implement automated testing and continuous delivery practices.
7. Design software that is secure, privacy-aware, observable, and resilient.
8. Evaluate the usefulness and limitations of an AI-assisted capability.
9. Collaborate effectively in a multidisciplinary software-development team.
10. Communicate product value and technical decisions to different stakeholders.

The project should integrate knowledge and techniques acquired in other courses of the programme, including software architecture, requirements engineering, software quality, secure development, DevOps, distributed systems, model-driven engineering, functional programming, and project management.

3 Product Scope

Each team must select:

- A clearly identified community
- A specific problem affecting that community
- A primary group of users or beneficiaries
- A measurable outcome that the product intends to improve

The proposed product should help users coordinate information, decisions, people, or resources before, during, or after a disruption.

Teams have freedom to define their product, target users, business model, and technical approach. However, the proposed scope must be achievable within the available project period.

4 Example Product Directions

The following examples are intended as inspiration only. Teams may propose other product directions, provided that the proposal addresses the central project challenge.

4.1 Campus Continuity Assistant

A platform that helps students, staff, and service teams respond to room closures, timetable changes, transport interruptions, equipment failures, and accessibility barriers.

Potential capabilities include:

- Personalized disruption notifications
- Alternative room or route suggestions
- Verified institutional announcements
- User-submitted reports
- Duplicate-report detection
- Multilingual summaries
- Offline access to essential information

4.2 Neighbourhood Mutual-Aid Platform

A product that matches temporary community needs with volunteers, organizations, or available resources.

Potential capabilities include:

- Requests and offers of assistance
- Volunteer coordination
- Resource reservations
- Trust and moderation workflows
- Privacy-preserving location information
- Recognition of verified contributions

4.3 Local Event Coordination Platform

A platform that supports organizers, service providers, volunteers, and participants before, during, and after community events.

Potential capabilities include:

- Capacity and queue reporting
- Accessibility information
- Lost-and-found workflows
- Incident triage
- Organizer dashboards

- Participant notifications
- Post-event analytics

4.4 Sustainable Resource-Sharing Platform

A product for sharing reusable goods, tools, food surpluses, transport resources, rooms, equipment, or other community assets.

Potential capabilities include:

- Resource listings
- Reservations and collection workflows
- Availability notifications
- Reliability indicators
- Environmental-impact estimates
- Misuse and abuse prevention

4.5 Accessible Journey and Service Planner

A platform that helps users plan journeys and access services while accounting for temporary or permanent accessibility constraints.

Potential capabilities include:

- Barrier reporting
- Accessible route suggestions
- Confidence and provenance indicators
- Voice interaction
- Plain-language explanations
- Notifications when conditions change

4.6 Community Energy Flexibility Assistant

A product that helps households or communities respond to consumption peaks, variable generation, service interruptions, or resource constraints.

Potential capabilities include:

- Consumption-data ingestion
- Configurable alerts
- Suggested actions
- Community goals
- Forecast explanations
- Consent and data-retention controls

5 Common Product Requirements

Although teams may develop different products, every project must include the following product-management activities.

5.1 Problem Discovery

The team must investigate and document:

- The target community
- The problem to be addressed
- The stakeholders affected by the problem
- Existing practices or alternative solutions
- The consequences of leaving the problem unresolved
- The principal assumptions behind the proposed product

5.2 Market and Competitor Analysis

The team must identify relevant competing products, services, or non-digital alternatives.

The analysis should consider:

- Target users
- Principal features
- Strengths and weaknesses
- Business or sustainability model
- User experience
- Privacy and trust
- Opportunities for differentiation

The team must use this analysis to define a clear value proposition.

5.3 User and Stakeholder Research

Teams should validate important assumptions through suitable research methods, which may include:

- Interviews
- Surveys
- Observation
- Prototype testing
- Analysis of public reports or datasets
- Review of documented user experiences
- Consultation with relevant stakeholders

Limitations in access to real users must be documented. Teams must not fabricate user-research evidence.

5.4 Product Vision and Success Criteria

Each team must define:

- Product vision
- Target users
- Value proposition
- Principal user journeys
- Expected community benefit
- MVP boundary
- Product risks
- Measurable success criteria

5.5 Ethical Engagement Design

Teams must design at least one mechanism intended to encourage constructive user participation.

Possible mechanisms include:

- Progress indicators
- Community goals
- Personal challenges
- Contribution histories
- Badges
- Collaborative activities
- Personalized reminders
- Recognition from beneficiaries
- Optional rankings or leaderboards

The team must explain:

1. Which behaviour the mechanism intends to encourage.
2. Why that behaviour benefits the user or community.
3. How misuse or manipulation is prevented.
4. Whether the mechanism could exclude, expose, or discourage users.
5. How users can control or opt out of the mechanism.

Points, badges, and leaderboards should not be introduced without a clear connection to the product's desired outcomes.

5.6 Responsible AI

Each team must implement at least one meaningful AI-assisted workflow. AI must address a demonstrated user or business need and must not be included solely to satisfy a technical requirement.

Suitable examples include:

- Classification and routing
- Summarization of multiple reports
- Identification of related or duplicate submissions
- Translation
- Plain-language rewriting
- Retrieval over verified community information
- Extraction of structured information
- Explanation of recommendations
- Generation of draft content for human approval

A general-purpose chatbot is not mandatory. Teams choosing a conversational interface must demonstrate why conversation is appropriate for the selected user journey.

5.6.1 AI Evaluation

The team must define how its AI-assisted capability will be evaluated.

The evaluation must include:

- Intended purpose
- Representative inputs
- Difficult or adverse cases
- Expected outputs
- Acceptance criteria
- Relevant quality measures
- Latency considerations
- Cost considerations, where applicable
- Known limitations

The evaluation dataset may be created by the team, but its origin, scope, and limitations must be documented.

5.6.2 Baseline and Fallback

The team must provide either:

- A non-AI baseline against which the AI capability can be compared, or

- A non-AI fallback that allows the principal workflow to continue when the AI capability is unavailable

The interface must not present uncertain AI-generated content as verified fact.

5.6.3 AI Safety and Trust

Where relevant, the solution should address:

- Inaccurate or fabricated outputs
- Prompt injection
- Inappropriate disclosure of information
- Harmful or unsuitable content
- Bias and unfair treatment
- Provenance of retrieved information
- User correction of AI-generated content
- Human approval for consequential actions

Teams must document the limitations of their AI component and explain how those limitations are communicated to users.

5.7 Investor Engagement

Teams must engage with the teaching staff as investors and product stakeholders.

At least one product or technical decision must be revised in response to investor feedback. The change and its rationale must be documented.

6 Common Technical Requirements

Each team has freedom to select the technology stack and architecture that best support its product. All significant technical decisions must be justified.

6.1 Client Application

The product must provide an appropriate user-facing application.

Possible approaches include:

- Cross-platform mobile application
- Native mobile application
- Responsive web application
- Progressive web application
- A combination of mobile and web interfaces

The interface must support the team's principal user journeys and should follow accessibility and usability principles.

6.2 Backend Architecture

The solution must contain at least two independently deployable backend components.

Teams must justify:

- Component or service boundaries
- Responsibilities and data ownership
- Synchronous and asynchronous communication
- Coupling and cohesion
- Deployment implications
- Failure handling

A modular monolith may be accepted when the team presents convincing evidence that independently deployable services would introduce unnecessary complexity. In that case, the internal module boundaries must remain explicit and enforceable.

6.3 Asynchronous Workflow

The solution must implement at least one meaningful asynchronous workflow.

Examples include:

- Processing and classifying a submitted report
- Generating notifications
- Enriching information through an AI service
- Updating analytics
- Expiring a reservation
- Processing external events
- Synchronizing data with another system

The workflow should demonstrate appropriate handling of retries, duplicate events, failures, and eventual consistency.

6.4 Data Management

The product must use persistent data storage appropriate to the selected problem.

The team must explain:

- Data entities and relationships
- Data ownership
- Consistency requirements
- Data lifecycle
- Retention and deletion
- Backup or recovery considerations

- Relevant privacy constraints

Teams should not introduce multiple database technologies unless the product requirements justify the additional complexity.

6.5 External Integration

The product must integrate with at least one external system or a realistic simulator of an external system.

Examples include:

- Transport information
- Weather observations
- Maps or geospatial services
- Institutional announcements
- Calendars
- Environmental sensors
- Energy-consumption data
- Facility-management systems

The application must handle unavailable, delayed, incomplete, or invalid external data appropriately.

6.6 Authentication and Authorization[1]

The solution must implement appropriate identity and access controls.

Depending on the product, this may include:

- User authentication
- Role-based or policy-based authorization
- Administrative access
- Service-to-service authentication
- Audit records
- Account and consent management

6.7 Observability

The deployed solution must provide sufficient information for the team to understand its operational state.

At minimum, the project should include:

- Structured application logs

[1] For the MVP it is acceptable that the authentication mechanism is mocked as to allow the team more time to tackle higher-value user journeys

- Health checks
- Relevant metrics
- Correlation or trace identifiers
- Error reporting
- A small operational dashboard

Teams should identify meaningful service-level indicators for their product.

6.8 Resilience and Degraded Operation

Each team must define and implement at least one degraded-mode scenario.

The product presentation must demonstrate what happens when one of the following is unavailable:

- An external data provider
- An AI service
- A network connection
- A database
- A notification provider
- A backend component

The product should fail safely and communicate limitations clearly to users.

7 Project Milestones

The project will be completed in three sprints.

7.1 Sprint 1: Discover, Validate, and Design

7.1.1 Objective

The objective of Sprint 1 is to establish that the team is addressing a meaningful problem and has proposed a credible product and technical direction.

7.1.2 Deliverables

1. **Problem and Opportunity Report**
    - Target community
    - Problem statement
    - Stakeholder map
    - Problem evidence
    - Assumptions and risks
2. **Market and Competitor Analysis**
    - Relevant alternatives

    - Strengths and weaknesses
    - Opportunity for differentiation
3. **User Research**
    - Research method
    - Findings
    - Limitations
    - Resulting product decisions
4. **Product Vision**
    - Personas
    - Value proposition
    - Principal user journeys
    - Success criteria
    - MVP definition
5. **Product Backlog**
    - Prioritized user stories or backlog items
    - Acceptance criteria
    - Sprint objectives
    - Initial release plan
6. **Responsible AI Opportunity Assessment**
    - Proposed AI-assisted workflow
    - Expected value
    - Risks
    - Evaluation approach
    - Baseline or fallback strategy
7. **Technical Design**
    - System context
    - Container or component view
    - Data model
    - API outline
    - Asynchronous workflow
    - External integration

- Deployment view

8. **Architecture Decision Records**
    - Principal architectural and technological choices
    - Alternatives considered
    - Decision rationale
    - Consequences and trade-offs
9. **Security and Privacy Assessment**
    - Initial threat model
    - Sensitive data
    - Access-control requirements
    - Data-minimization measures
    - Retention considerations
10. **Walking Skeleton**
    - Minimal end-to-end implementation
    - Source repository
    - Automated build
    - Initial test
    - Containerized deployment

7.1.3 Investor Review

The Sprint 1 review should answer:

Is this a valuable problem, and is the proposed MVP the smallest credible product experiment?

7.2 Sprint 2: Build and Operate the Vertical Slice

7.2.1 Objective

The objective of Sprint 2 is to deliver an operational end-to-end version of the product's most important user journey.

7.2.2 Deliverables

1. Functional client application
2. Authentication and authorization
3. Persistent data storage
4. Documented API contracts
5. At least two backend components, or an approved modular alternative

6. One meaningful asynchronous workflow
7. Initial external-system integration
8. First version of the AI-assisted capability
9. Non-AI baseline or fallback
10. Automated unit, integration, and relevant end-to-end tests
11. CI/CD pipeline
12. Structured logging, health checks, and initial metrics
13. Demonstration of at least one failure condition
14. Updated backlog and architecture documentation

7.2.3 Investor Review

The Sprint 2 review should answer:

Does the product already deliver observable user value, and can the team deploy, monitor, and operate it?

7.3 Sprint 3: Evaluate, Harden, and Present

7.3.1 Objective

The objective of Sprint 3 is to refine the MVP, evaluate its most important qualities, and demonstrate that it is a credible and maintainable product.

7.3.2 Deliverables

1. Refined MVP incorporating stakeholder feedback
2. Completed principal user journeys
3. AI evaluation results
4. Usability evaluation or testing plan
5. Accessibility review
6. Security review and updated threat model
7. Performance and scalability evidence
8. Resilience and degraded-mode demonstration
9. Operational dashboard
10. Updated architecture views and decision records
11. Architecture evolution narrative
12. Known limitations and technical-debt register
13. Product roadmap
14. Final investor pitch

15. Final technical demonstration
16. Individual contribution evidence

7.3.3 Investor Review

The final review should answer:

Is this a credible product and a maintainable software system worth further investment?

8 Development and Delivery Practices

8.1 Version Control

Teams must use Git for version control.

The repository should demonstrate professional collaborative practices, including:

- Meaningful commits
- Branching strategy
- Pull requests
- Code reviews
- Issue or backlog references
- Suitable repository documentation

Raw commit counts will not be treated as reliable evidence of contribution.

8.2 Agile Project Management

Teams must use Jira or an equivalent project-management platform to:

- Manage the product backlog
- Plan sprints
- Assign and track work
- Record decisions
- Document sprint goals
- Monitor progress
- Record stakeholder feedback

8.3 Testing

The test strategy should be proportional to the system's risks and architecture.

It may include:

- Unit testing
- Integration testing
- Contract testing

- End-to-end testing
- Security testing
- Performance testing
- Resilience testing
- AI evaluation tests

Teams must justify the types of tests selected and identify important areas that remain untested.

8.4 Containerization and Deployment

The application must be delivered using containers.

Teams should provide:

- Reproducible builds
- Environment-specific configuration
- Secure secret handling
- Deployment instructions
- Automated deployment where feasible
- A documented rollback or recovery approach

The use of container orchestration is encouraged when justified by the architecture and deployment requirements.

8.5 Continuous Integration and Delivery

The project must include an automated pipeline that performs relevant activities such as:

- Compilation or packaging
- Automated tests
- Static analysis
- Dependency or security checks
- Container-image creation
- Deployment to a suitable environment

The pipeline should provide rapid and visible feedback when a change introduces a defect.

9 Documentation

Documentation should remain concise, current, and directly connected to the delivered product.

The final project documentation should include:

- Product vision
- Personas and stakeholder map

- Product backlog
- Requirements and quality attributes
- Architecture views
- Architecture Decision Records
- API documentation
- Data model
- Threat model
- AI evaluation report
- Test strategy and evidence
- Deployment and operation guide
- Known limitations
- Product roadmap
- Individual contribution evidence

Teams should prefer documentation generated from or maintained close to the source code where appropriate.

10 Evaluation Criteria

The project is completed as a team, but evaluation is individual. The final assessment considers both the quality of the team's product and the demonstrated contribution, understanding, collaboration, and professional conduct of each student.

Each student must be able to explain:

- The product vision
- The principal user journeys
- The overall architecture
- The responsibilities of the main components
- Important technical decisions
- Security and privacy risks
- The AI evaluation approach
- The deployment and operational model
- Their individual contributions

Evidence of individual contribution may include:

- Authored or co-authored pull requests
- Code reviews

- Tests
- Architecture decisions
- Product-research activities
- Backlog refinement
- Deployment work
- Operational improvements
- Documentation
- Sprint-review participation
- Individual reflections

Assessment will focus on the quality and impact of contributions, not merely their quantity.

11 Ethical and Professional Conduct

Teams must follow ethical and professional practices throughout the project.

In particular:

- Personal data must not be collected without a justified purpose.
- Real personal or sensitive data should not be used when synthetic or anonymized data is sufficient.
- User-research participants must understand the purpose of the research.
- Research results must not be fabricated or misrepresented.
- AI-generated content must be reviewed before being treated as authoritative.
- Security controls must not be disabled solely to simplify the demonstration.
- Third-party code, datasets, tools, and generated content must be acknowledged appropriately.
- Team members must engage respectfully and contribute professionally.

12 Final Submission

The final submission must provide sufficient information for the teaching staff to inspect, deploy, execute, and evaluate the product.

It should include:

1. Source-code repository
2. Release version of the product
3. Deployment instructions
4. Architecture documentation
5. Product and requirements documentation

6. API documentation
7. Test and quality evidence
8. AI evaluation report
9. Security and privacy assessment
10. Operational dashboard
11. Product roadmap
12. Final presentation
13. Individual contribution statements

Any functionality that cannot be executed in the evaluation environment must be clearly identified, explained, and demonstrated through suitable evidence.

13 Closing Challenge

At the end of the project, each team should be able to make a convincing case that:

The team has identified a real community problem, built the smallest credible product that addresses it, evaluated its most important assumptions, and delivered a secure, responsible, observable, and maintainable software system.